

Root Cause Analysis & Resolution Guide

Project: LifeGuard Insurance Platform (CI/CD Pipeline Enhancement)

1. Executive Summary & Issue Description

During the implementation of the Jenkins declarative CI/CD pipeline for the `lifeguard-insurance` platform, multiple critical build execution failures were encountered during the Maven compilation stage (`mvn clean package`). This run-book isolates the root causes of those structural, environmental, and syntax errors, and provides definitive remediation workflows for engineering alignment.

Primary Console Error Signature:

```
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:
3.13.0:compile (default-compile) on project lifeguard-insurance: Fatal error
compiling: error: release version 21 not supported
```

2. Root Cause Analysis (RCA)

Investigation into the target build node uncovered two independent bottlenecks working in tandem to break the build cycle:

- **Missing Compiler Binary (JDK vs. JRE Mismatch):** While the system target host successfully executed `java -version` pointing to an OpenJDK 21 execution context, the underlying infrastructure lacked the Java Development Kit compiler framework (`javac`). The host was provisioned with a headless Java Runtime Environment (JRE), which lacks structural mechanisms required by the `maven-compiler-plugin` to parse and package target definitions.
- **XML Configuration Syntax Violation:** An effort to pass a legacy feature parameter (`--enable-preview`) inside the project's `pom.xml` configurations resulted in a fatal build parse exception. Standard XML limits the parsing of double-dashes (`--`) inside continuous comment segments (`<!-- -->`), prompting an automatic model parse break.

3. Comprehensive Step-by-Step Resolution Workflows

Step 3.1: Host Machine Infrastructure Remediation

To transition the target host from a passive JRE state to an active JDK structural compilation tier, the underlying OpenJDK developer binaries must be updated explicitly via the native packet manager.

Execute the following environment initialization steps on the target Ubuntu build node machine:

```
# Update local package definitions repositories
sudo apt update

# Install the explicit OpenJDK 21 Developer compiler kit
sudo apt install -y openjdk-21-jdk

# Verify system mapping paths find a valid compiler entrypoint
readlink -f $(which javac)
```

Expected Valid Output Path: `/usr/lib/jvm/java-21-openjdk-amd64/bin/javac`

Step 3.2: Code-Level Configuration Optimization

Open the project's root **pom.xml** and ensure the `<build>` plug-ins mirror the configuration pattern detailed below. This configuration cleanly excises the invalid comment strings, locks down plugin version boundaries, and targets standard stable Java 21 properties.

Refactored Plug-in Structural Segment (pom.xml):

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
      <configuration>
        <source>21</source>
        <target>21</target>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
```

Production Best Practice Reminder:

Always verify configuration states are pushed cleanly to your source repository before signaling a rebuild execution to the pipeline:

```
git add pom.xml && git commit -m "fix: resolve compiler flags" && git push origin main
```

Step 3.3: Jenkins Global Tool Alignment Integration

To protect pipeline workers from inheriting arbitrary environment paths from random host instances, register the newly verified JDK home as a permanent managed tool token.

1. Navigate to the **Jenkins Web Dashboard**.
2. Select **Manage Jenkins** → **Tools** (or *Global Tool Configuration*).
3. Locate the **JDK Configurations** interface block and click **Add JDK**.
4. Assign the definitive indexing name identifier token: `jdk21`.
5. Uncheck the *Install automatically* selection option.
6. Input the explicit home path string in the **JAVA_HOME** text container field: `/usr/lib/jvm/java-21-openjdk-amd64`.
7. Apply and click **Save**.

4. Standardized Pipeline Reference Implementation

The following snippet provides an isolated, production-grade Declarative Jenkinsfile schema leveraging the registered `jdk21` global environment wrapper toolchain to guarantee consistent build execution:

```

pipeline {
    agent any

    tools {
        // Enforces Jenkins to map the workspace environment to our verified JDK 21 paths
        jdk 'jdk21'
    }

    stages {
        stage('1. Synchronize Workspace SCM') {
            steps {
                echo "Pulling optimized code branch from enterprise source
infrastructure..."
                git branch: 'main', url: 'https://github.com/AjaySinghChauhan85/LifeGuard-
Insurance_DevOps.git'
            }
        }

        stage('2. Execute Maven Compilation Assembly') {
            steps {
                sh '''
                    echo "--- Toolchain Runtime Integrity Verification ---"
                    java -version
                    mvn -version

                    echo "--- Initiating Production Target Build ---"
                    mvn clean package -DskipTests
                    echo "Target lifecycle phase package operation successfully
concluded!"
                '''
            }
        }
    }
}

```